

Đây là notebook đi kèm cho cuốn sách [Deep Learning with Python, Third Edition](#). Để dễ đọc, nó chỉ chứa các khối mã có thể chạy được và tiêu đề các phần, đồng thời lược bỏ mọi nội dung khác trong sách: các đoạn văn bản, hình minh họa và mã giả.

Nếu bạn muốn theo dõi những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản sao cuốn sách của mình.

Nội dung của cuốn sách có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "tensorflow"
```

[Hiện mã](#)

- ✓ Các khối xây dựng toán học của mạng thần kinh
- ✓ Cái nhìn đầu tiên về một mạng thần kinh

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
train_images.shape
```

```
len(train_labels)
```

```
train_labels
```

```
test_images.shape
```

```
len(test_labels)
```

```
test_labels
```

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense(512, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
test_digits = test_images[0:10]
predictions = model.predict(test_digits)
predictions[0]
```

```
predictions[0].argmax()
```

```
predictions[0][7]
```

```
test_labels[0]
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"test_acc: {test_acc}")
```

✓ Biểu diễn dữ liệu cho mạng thần kinh

✓ Scalars (Tensor hạng 0)

```
import numpy as np
x = np.array(12)
x
```

```
x.ndim
```

✓ Vectors (Tensor hạng 1)

```
x = np.array([12, 3, 6, 14, 7])
x
```

```
x.ndim
```

✓ Matrices (Tensor hạng 2 - Ma trận)

```
x = np.array([[5, 78, 2, 34, 0],
              [6, 79, 3, 35, 1],
              [7, 80, 4, 36, 2]])
x.ndim
```

✓ Rank-3 tensors và các tensor hạng cao hơn

```
x = np.array([[[5, 78, 2, 34, 0],
               [6, 79, 3, 35, 1],
               [7, 80, 4, 36, 2]],
              [[5, 78, 2, 34, 0],
               [6, 79, 3, 35, 1],
               [7, 80, 4, 36, 2]],
              [[5, 78, 2, 34, 0],
               [6, 79, 3, 35, 1],
               [7, 80, 4, 36, 2]]])
x.ndim
```

✓ Các thuộc tính chính

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
train_images.ndim
```

```
train_images.shape
```

```
train_images.dtype
```

```
import matplotlib.pyplot as plt
```

```
digit = train_images[4]
plt.imshow(digit, cmap=plt.cm.binary)
plt.show()
```

```
train_labels[4]
```

▼ Thao tác với tensor trong NumPy

```
my_slice = train_images[10:100]
my_slice.shape
```

```
my_slice = train_images[10:100, :, :]
my_slice.shape
```

```
my_slice = train_images[10:100, 0:28, 0:28]
my_slice.shape
```

```
my_slice = train_images[:, 14:, 14:]
```

```
my_slice = train_images[:, 7:-7, 7:-7]
```

▼ Khái niệm về các lô dữ liệu (data batches)

```
batch = train_images[:128]
```

```
batch = train_images[128:256]
```

```
n = 3
batch = train_images[128 * n : 128 * (n + 1)]
```

▼ Ví dụ thực tế về các tensor dữ liệu

Dữ liệu Vector

Dữ liệu chuỗi thời gian (Timeseries) hoặc dữ liệu trình tự (Sequence)

Dữ liệu hình ảnh

Dữ liệu Video

▼ Các bánh răng của mạng thần kinh: Các phép toán Tensor

▼ Các phép toán trên từng phần tử (Element-wise operations)

```
def naive_relu(x):
    assert len(x.shape) == 2
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

```
def naive_add(x, y):
    assert len(x.shape) == 2
    assert x.shape == y.shape
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] += y[i, j]
    return x
```

```
import time

x = np.random.random((20, 100))
y = np.random.random((20, 100))

t0 = time.time()
for _ in range(1000):
    z = x + y
    z = np.maximum(z, 0.0)
print("Took: {:.2f} s".format(time.time() - t0))
```

```
t0 = time.time()
for _ in range(1000):
    z = naive_add(x, y)
    z = naive_relu(z)
print("Took: {:.2f} s".format(time.time() - t0))
```

▼ Lan truyền (Broadcasting)

```
import numpy as np

X = np.random.random((32, 10))
y = np.random.random((10,))
```

```
y = np.expand_dims(y, axis=0)
```

```
Y = np.tile(y, (32, 1))
```

```
def naive_add_matrix_and_vector(x, y):
    assert len(x.shape) == 2
    assert len(y.shape) == 1
    assert x.shape[1] == y.shape[0]
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] += y[j]
    return x
```

```
import numpy as np

x = np.random.random((64, 3, 32, 10))
y = np.random.random((32, 10))
z = np.maximum(x, y)
```

▼ Tích Tensor (Tensor product)

```
x = np.random.random((32,))
y = np.random.random((32,))

z = np.matmul(x, y)
z = x @ y
```

```
def naive_vector_product(x, y):
    assert len(x.shape) == 1
    assert len(y.shape) == 1
    assert x.shape[0] == y.shape[0]
    z = 0.0
    for i in range(x.shape[0]):
        z += x[i] * y[i]
    return z
```

```
def naive_matrix_vector_product(x, y):
    assert len(x.shape) == 2
    assert len(y.shape) == 1
    assert x.shape[1] == y.shape[0]
    z = np.zeros(x.shape[0])
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
```

```
        z[i] += x[i, j] * y[j]
    return z
```

```
def naive_matrix_vector_product(x, y):
    z = np.zeros(x.shape[0])
    for i in range(x.shape[0]):
        z[i] = naive_vector_product(x[i, :], y)
    return z
```

```
def naive_matrix_product(x, y):
    assert len(x.shape) == 2
    assert len(y.shape) == 2
    assert x.shape[1] == y.shape[0]
    z = np.zeros((x.shape[0], y.shape[1]))
    for i in range(x.shape[0]):
        for j in range(y.shape[1]):
            row_x = x[i, :]
            column_y = y[:, j]
            z[i, j] = naive_vector_product(row_x, column_y)
    return z
```

▼ Thay đổi hình dạng Tensor (Tensor reshaping)

```
train_images = train_images.reshape((60000, 28 * 28))
```

```
x = np.array([[0., 1.],
              [2., 3.],
              [4., 5.]])
x.shape
```

```
x = x.reshape((6, 1))
x
```

```
x = x.reshape((2, 3))
x
```

```
x = np.zeros((300, 20))
x = np.transpose(x)
x.shape
```

Diễn giải hình học của các phép toán tensor

Diễn giải hình học của deep learning

▼ Động cơ của mạng thần kinh: Tối ưu hóa dựa trên gradient

Đạo hàm là gì?

Đạo hàm của một phép toán tensor: Gradient

Hạ gradient ngẫu nhiên (Stochastic gradient descent)

▼ Chuỗi đạo hàm: Thuật toán Lan truyền ngược (Backpropagation)

Quy tắc chuỗi (The chain rule)

Tính đạo hàm tự động với đồ thị tính toán

▼ Nhìn lại ví dụ đầu tiên của chúng ta

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

```
model = keras.Sequential(
    [
        layers.Dense(512, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

```
model.fit(
    train_images,
    train_labels,
    epochs=5,
    batch_size=128,
)
```

✓ Tự triển khai lại ví dụ đầu tiên từ đầu (from scratch)

✓ Một lớp Dense đơn giản

```
import keras
from keras import ops

class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation
        self.W = keras.Variable(
            shape=(input_size, output_size), initializer="uniform"
        )
        self.b = keras.Variable(shape=(output_size,), initializer="zeros")

    def __call__(self, inputs):
        x = ops.matmul(inputs, self.W)
        x = x + self.b
        if self.activation is not None:
            x = self.activation(x)
        return x

    @property
    def weights(self):
        return [self.W, self.b]
```

✓ Một lớp Sequential đơn giản

```
class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs
        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def weights(self):
        weights = []
        for layer in self.layers:
            weights += layer.weights
        return weights
```

```

model = NaiveSequential(
    [
        NaiveDense(input_size=28 * 28, output_size=512, activation=ops.relu),
        NaiveDense(input_size=512, output_size=10, activation=ops.softmax),
    ]
)
assert len(model.weights) == 4

```

▼ Một trình tạo lô (Batch generator)

```

import math

class BatchGenerator:
    def __init__(self, images, labels, batch_size=128):
        assert len(images) == len(labels)
        self.index = 0
        self.images = images
        self.labels = labels
        self.batch_size = batch_size
        self.num_batches = math.ceil(len(images) / batch_size)

    def next(self):
        images = self.images[self.index : self.index + self.batch_size]
        labels = self.labels[self.index : self.index + self.batch_size]
        self.index += self.batch_size
        return images, labels

```

▼ Chạy một bước huấn luyện

▼ Bước cập nhật trọng số

```

learning_rate = 1e-3

def update_weights(gradients, weights):
    for g, w in zip(gradients, weights):
        w.assign(w - g * learning_rate)

```

```

from keras import optimizers

optimizer = optimizers.SGD(learning_rate=1e-3)

def update_weights(gradients, weights):
    optimizer.apply_gradients(zip(gradients, weights))

```

▼ Tính toán Gradient

```

%%backend tensorflow
import tensorflow as tf

x = tf.zeros(shape=())
with tf.GradientTape() as tape:
    y = 2 * x + 3
grad_of_y_wrt_x = tape.gradient(y, x)

```

```

%%backend tensorflow
def one_training_step(model, images_batch, labels_batch):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)
        loss = ops.sparse_categorical_crossentropy(labels_batch, predictions)
        average_loss = ops.mean(loss)
    gradients = tape.gradient(average_loss, model.weights)
    update_weights(gradients, model.weights)
    return average_loss

```

▼ Vòng lặp huấn luyện đầy đủ

```

%%backend tensorflow
def fit(model, images, labels, epochs, batch_size=128):

```

```
for epoch_counter in range(epochs):
    print(f"Epoch {epoch_counter}")
    batch_generator = BatchGenerator(images, labels)
    for batch_counter in range(batch_generator.num_batches):
        images_batch, labels_batch = batch_generator.next()
        loss = one_training_step(model, images_batch, labels_batch)
        if batch_counter % 100 == 0:
            print(f"loss at batch {batch_counter}: {loss:.2f}")
```

Đánh giá mô hình

▼ Evaluating the model

```
%%backend tensorflow
predictions = model(test_images)
predicted_labels = ops.argmax(predictions, axis=1)
matches = predicted_labels == test_labels
f"accuracy: {ops.mean(matches):.2f}"
```